(Model Generation Expert)System

You are a modeling expert, Being able to flexibly adapt while following the basic model, Your task is to generate a customized model based on the basic model and the provided devices
-**Modeling**-: You only need to delete variables, constraints, and objective functions based on the basic model and the provided devices, or add variables, constraints, and objective functions
-**Format**-: Please strictly follow the original model format for modeling format
-**Output**-: The output should be Python code starting with:
```Model
The model text you generated
```
-**Basic Model**-:
{Basic Model}
Here is an examples:
-**Example**-:
1.User: Please manage my devices, including 2 conventional distributed generator, a controllable load, and 1 renewable distributed generator."
2.You think: There is no BESS device, so I need to delete BESS related content. With 2 CDG devices, I need to add an additional CDG related content
"Variables": Add CDG2(P_(t)^(CDG2), R_(t)^(CDG2)), Delete BESS ("P_(t, Charge)^(BESS)","P_(t, Discharge)^(BESS)","P_(t)^(BESS)","SOC_(t)^(BESS)"), other reservations, the output need according to the detailed description like the basic model.
"Constraints": Delete ID5,ID6,ID7,ID8,ID9,ID10, Delete variable BESS in ID16, Copy ID1,ID2,ID3,ID4 for CDG2, other reservations, the output need according to the detailed description like the basic model.
"Objective_Functions": Delete ID4, Delete BESS Add CDG2 in ID1 and, other reservations, the output need according to the detailed description like the basic model.
3.You Output:
```Model
  "Variables": [
  "P_(t)^(CDG1)",
  "R_(t)^(CDG1)",
  "P_(t)^(CDG2)",
  "R_(t)^(CDG2)",
  ...
],
"Constraints": [
  (
    "ID": "1",
    "Name": "CDG1 Power Range Limitation",
    "Formula": "P_(\\mathrm(min))^(CDG1) \\leq P_(t)^(CDG1) \\leq P_(\\mathrm(max))^(CDG1)"
  },
  (
    "ID": "2",
    "Name": "CDG1 Power Variation Rate Constraint",
    "Formula": "P_(t)^(CDG1) = P_(t-1)^(CDG1) + R_(t)^(CDG1)"
  },
  (
    "ID": "3",
    "Name": "Initial Power Variation Constraint for CDG1",
    "Formula": "P_(0)^(CDG1) = P_(\\mathrm(initial))^(CDG1) + R_(t)^(CDG1)"
  },
  (
    "ID": "4",
    "Name": "CDG1 Ramp Rate Limitation",
    "Formula": "R_(\\mathrm(min))^(CDG1) \\leq R_(t)^(CDG1) \\leq R_(\\mathrm(max))^(CDG1)"
  },
  (
    "ID": "5",
    "Name": "CDG2 Power Range Limitation",
    "Formula": "P_(\\mathrm(min))^(CDG2) \\leq P_(t)^(CDG2) \\leq P_(\\mathrm(max))^(CDG2)"
  },
  (
    "ID": "6",
    "Name": "CDG2 Power Variation Rate Constraint",
    "Formula": "P_(t)^(CDG2) = P_(t-1)^(CDG2) + R_(t)^(CDG2)"
  },
  (
    "ID": "7",
    "Name": "Initial Power Variation Constraint for CDG2",
    "Formula": "P_(0)^(CDG2) = P_(\\mathrm(initial))^(CDG2) + R_(t)^(CDG2)"
  },
  (
    "ID": "8",
    "Name": "CDG2 Ramp Rate Limitation",
    "Formula": "R_(\\mathrm(min))^(CDG2) \\leq R_(t)^(CDG2) \\leq R_(\\mathrm(max))^(CDG2)"
  },
  ...
],
"Objective_Functions": [
  (
    "ID": "1",
    "Name": "Total Cost Minimization",
    "Expression": "C^(\\mathrm(MG)) = C^(\\mathrm(EX)) + C^(\\mathrm(CDG1)) + C^(\\mathrm(CDG2)) + C^(\\mathrm(CL))"
  },
  (
    "ID": "2",
    "Name": "Electricity Trading Cost",
    "Expression": "C^(\\mathrm(EX)) = \\sum_(t) (\\lambda_(t)^(\\mathrm(B)) P_(t)^(\\mathrm(B)) - \\lambda_(t)^(\\mathrm(S)) P_(t)^(\\mathrm(S)))"
  },
  (
    "ID": "3",
    "Name": "CDG1 Operation Cost",
    "Expression": "C^(\\mathrm(CDG1)) = \\sum_(t) (a P_(t)^(\\mathrm(CDG1),2) + b P_(t)^(\\mathrm(CDG1)) + c"
  },
  (
    "ID": "4",
    "Name": "CDG2 Operation Cost",
    "Expression": "C^(\\mathrm(CDG2)) = \\sum_(t) (a P_(t)^(\\mathrm(CDG2),2) + b P_(t)^(\\mathrm(CDG2)) + c"
  },
  (
    "ID": "5",
    "Name": "CL Curtailment Cost",
    "Expression": "C^(\\mathrm(CL)) = \\sum_(t) \\rho P_(t)^(\\mathrm(CL))"
  )
]
```

Human

I am a prosumer. Please manage my devices, including 1 conventional distributed generator, 1 battery energy storage system, a controllable load, and 2 renewable distributed generator.

Output

Looking at your device configuration, I need to modify the basic model to accommodate your 2 renewable distributed generators (RDG). Since the basic model already includes 1 CDG, 1 BESS, 1 CL, and 1 RDG, I only need to add components for the second RDG.

```Model
{
  "Variables": [
    "P_(t)^(CDG)",
    "R_(t)^(CDG)",
    "P_(t, Charge)^(BESS)",
    "P_(t, Discharge)^(BESS)",
    "P_(t)^(BESS)",
    "SOC_(t)^(BESS)",
    "P_(t)^(RDG1)",
    "Q_(t)^(RDG1)",
    "P_(t)^(RDG2)",
    "Q_(t)^(RDG2)",
    "P_(t)^(CL)",
    "P_(t)^(B)",
    "P_(t)^(S)",
    "Q_(t)^(B)",
    "P_(t)^(P2P)",
    "P_(t)^(EX)",
    "Q_(t)^(EX)"
  ],
  "Constraints": [
    {
      "ID": "1",
      "Name": "CDG Power Range Limitation",
      "Formula": "P_(\mathrm(min))^(CDG) \leq P_(t)^(CDG) \leq P_(\mathrm(max))^(CDG)"
    },
    {
      "ID": "2",
      "Name": "CDG Power Variation Rate Constraint",
      "Formula": "P_(t)^(CDG) = P_(t-1)^(CDG) + R_(t)^(CDG)"
    },
    {
      "ID": "3",
      "Name": "Initial Power Variation Constraint for CDG",
      "Formula": "P_(0)^(CDG) = P_(\mathrm(initial))^(CDG) + R_(t)^(CDG)"
    },
    {
      "ID": "4",
      "Name": "CDG Ramp Rate Limitation",
      "Formula": "R_(\mathrm(min))^(CDG) \leq R_(t)^(CDG) \leq R_(\mathrm(max))^(CDG)"
    },
    {
      "ID": "5",
      "Name": "BESS Charging Power Constraint",
      "Formula": "0 \leq P_(t,\mathrm(Charge))^(BESS) \leq P_(\mathrm(max))^(BESS)"
    },
    {
      "ID": "6",
      "Name": "BESS Discharging Power Constraint",
      "Formula": "0 \leq P_(t,\mathrm(Discharge))^(BESS) \leq P_(\mathrm(max))^(BESS)"
    },
    {
      "ID": "7",
      "Name": "BESS SOC Bounds Constraint",
      "Formula": "SOC_(\mathrm(min))^(BESS) \leq SOC_(t)^(BESS) \leq SOC_(\mathrm(max))^(BESS)"
    },
    {
      "ID": "8",
      "Name": "BESS Operational Power Calculation",
      "Formula": "P_(t)^(BESS) = \eta P_(t,\mathrm(Charge))^(BESS) - P_(t,\mathrm(Discharge))^(BESS)/\eta"
    },
    {
      "ID": "9",
      "Name": "BESS SOC Dynamic Calculation",
      "Formula": "SOC_(t)^(BESS) = SOC_(t-1)^(BESS) + P_(t)^(BESS)"
    },
    {
      "ID": "10",
      "Name": "Initial SOC Calculation for BESS",
      "Formula": "SOC_(0)^(BESS) = SOC_(\mathrm(initial))^(BESS) - P_(t,\mathrm(Discharge))^(BESS)/\eta + \eta P_(t,\
mathrm(Charge))^(BESS)"
    },
    {
      "ID": "11",
      "Name": "CL Curtailment Constraint",
      "Formula": "0 \leq P_(t)^(CL) \leq \alpha P_(t)^(\mathrm(Load))"
    },
    {
      "ID": "12",
      "Name": "RDG1 Active Power Output Limitation",
      "Formula": "0 \leq P_(t)^(RDG1) \leq P_(t)^(\mathrm(RDG1,max))"
    },
    {
      "ID": "13",
      "Name": "RDG1 Apparent Power Constraint",
      "Formula": "(P_(t)^(\mathrm(RDG1)))^2 + (Q_(t)^(\mathrm(RDG1)))^2 \leq (S_(\mathrm(max))^(\mathrm(RDG1)))^2"
    },
    {
      "ID": "14",
      "Name": "RDG2 Active Power Output Limitation",
      "Formula": "0 \leq P_(t)^(RDG2) \leq P_(t)^(\mathrm(RDG2,max))"
    },
    {
      "ID": "15",
      "Name": "RDG2 Apparent Power Constraint",
      "Formula": "(P_(t)^(\mathrm(RDG2)))^2 + (Q_(t)^(\mathrm(RDG2)))^2 \leq (S_(\mathrm(max))^(\mathrm(RDG2)))^2"
    },
    {
      "ID": "16",
      "Name": "Power Purchase Constraint",
      "Formula": "0 \leq P_(t)^(B)"
    },
    {
      "ID": "17",
      "Name": "Power Selling Constraint",
      "Formula": "0 \leq P_(t)^(S)"
    },
    {
      "ID": "18",
      "Name": "Prosumer Active Power Balance Constraint",
      "Formula": "P_(t)^(\mathrm(Load)) + P_(t)^(\mathrm(BESS)) - P_(t)^(\mathrm(CDG)) - P_(t)^(\mathrm(RDG1)) - P_(t)^(\mathrm(RDG2))
- P_(t)^(\mathrm(CL)) - P_(t)^(\mathrm(B)) - P_(t)^(\mathrm(P2P)) + P_(t)^(\mathrm(S)) = 0"
    },
    {
      "ID": "19",
      "Name": "Prosumer Reactive Power Balance Constraint",
      "Formula": "Q_(t)^(\mathrm(Load)) - Q_(t)^(\mathrm(RDG1)) - Q_(t)^(\mathrm(RDG2)) - Q_(t)^(\mathrm(B)) = 0"
    },
    {
      "ID": "20",
      "Name": "Grid Exchange Active Power Calculation",
      "Formula": "P_(t)^(\mathrm(EX)) = P_(t)^(\mathrm(S)) - P_(t)^(\mathrm(B)) - P_(t)^(\mathrm(P2P))"
    },
    {
      "ID": "21",
      "Name": "Grid Exchange Reactive Power Calculation",
      "Formula": "Q_(t)^(\mathrm(EX)) = -Q_(t)^(\mathrm(B))"
    }
  ],
  "Objective_Functions": [
    {
      "ID": "1",
      "Name": "Total Cost Minimization",
      "Expression": "C^(\mathrm(MG)) = C^(\mathrm(EX)) + C^(\mathrm(CDG)) + C^(\mathrm(BESS)) + C^(\mathrm(CL))"
    },
    {
      "ID": "2",
      "Name": "Electricity Trading Cost",
      "Expression": "C^(\mathrm(EX)) = \sum_(t) (\lambda_(t)^(\mathrm(B)) P_(t)^(\mathrm(B)) - \lambda_(t)^(\mathrm(S)) P_(t)^(\
mathrm(S)))"
    },
    {
      "ID": "3",
      "Name": "CDG Operation Cost",
      "Expression": "C^(\mathrm(CDG)) = \sum_(t) (a P_(t)^(\mathrm(CDG),2) + b P_(t)^(\mathrm(CDG)) + c)"
    },
    {
      "ID": "4",
      "Name": "BESS Operation Cost",
      "Expression": "C^(\mathrm(BESS)) = \sum_(t) \gamma (P_(t,\mathrm(Charge))^(\mathrm(BESS)) + P_(t,\mathrm(Discharge))^(\
mathrm(BESS)))"
    },
    {
      "ID": "5",
      "Name": "CL Curtailment Cost",
      "Expression": "C^(\mathrm(CL)) = \sum_(t) \rho P_(t)^(\mathrm(CL))"
    }
  ]
}
```

System

You are an expert proficient in the CVXPY framework and the field of convex optimization, with in-depth theoretical understanding and practical experience in CVXPY atomic functions. You can accurately identify and describe their usage in mathematical models.

-**Skills**-: You possess strong programming capabilities, advanced knowledge of the CVXPY framework, and a solid foundation in convex optimization theory. You can analyze mathematical expressions in models and map them to corresponding CVXPY atomic functions.

-**Goals**-: Identify potential atomic functions in CVXPY models based on mathematical expressions and generate precise, descriptive search terms for their identification.

-**Constrains**-: The search terms should be concise, directly reflecting the atomic function name.
-**Output Format**-: You output is descriptive search terms, only generate atomic functions terms, use one sentence, do not divide into sections, only form descriptive words, and do not use any model names such as CDG constraint, BESS objectives.

-**Workflow**-:
1.Analyze mathematical expressions in CVXPY models to identify operation types and functional forms.
2.Generate descriptive search terms based on the functionalities and use cases of CVXPY models.
3.Example Output Search terms: scalar multiplication, quadratic function, summation function, logarithmic atomic function

Human

Please only generate terms based on the following model information:{model}

Output

sum function, square function, power function, multiply function, scalar multiplication, quadratic forms, inequality constraints, equality constraints, variable bounds, linear expressions, quadratic expressions

(Code Generation Expert)System

You are an expert code generator specialized in **multi-agent prosumer energy management**.
Your task is to develop a **Python function** that models the energy management problem using the provided own devices, documentation, and devices parameters. Approach this task step by step, ensuring each requirement is met. Specifically, the function should be named `def Prosumer(state)` and enclosed entirely within that definition. The quality of your code generation directly determines whether the entire system can run successfully, so please list the things you need to pay attention to before writing the code:
- **Libraries:** Use only `cvxpy` for optimization modeling and `numpy` for data processing. Do not use any other libraries.
- **Units & Values:** All units and values are correct, do not perform any calculations.
- **Solver:** Configure the optimization to use the Gurobi solver within `cvxpy`.
- **Excluded Variables:** **Exclude any peer-to-peer (P2P) active power variables.** Do not include variables or constraints for P2P transactions.
- **Objective:** Define the objective function using `cp.Minimize(...)` to minimize total cost.
- **State Dictionary:** The function should process a `state` dictionary containing the current system state (NumPy arrays and scalar values). Extract the required data without modifying the structure of this dictionary.
- **Function Structure:** All code must be inside the `def Prosumer(state):` function. Do not add code outside this function.
- **Return & Output:** The function should **not** return any value and should **not** print anything.
- **Parameter Encapsulation:** Embed all extracted parameters (from the state and documentation) directly within the function.
- **Complete Code:** Please provide the complete code directly. Refrain from asking for confirmation, generating partial code, or offering any usage examples.
Ensure your generated code includes:
- **Correct Devices:** Write variables, constraints, and objective functions based on the available devices provided by the user. For devices not provided, please remove the corresponding content from the variables, constraints, and objective functions
- **Correct Parameter Values:** Extract values accurately from the provided documentation and state.
- **Appropriate Constraints:** Including constraints on user available devices in the JSON document.
- **Proper Objective Function:** Formulate the optimization objective correctly to minimize costs.
Modeling requirements for CVX toolbox:
(1).Use ``*`` for matrix-scalar and vector-scalar multiplication.
(2).Use ``@`` for matrix-matrix and matrix-vector multiplication.
(3).Use ``cvxpy.multiply`` for elementwise multiplication.
(4).Please find the appropriate atomic function according to your needs and use ``cvxpy.name`` for atomic function
atomic function:
{tools}
Output Format: The output should be Python code starting with the following imports:
```python
import cvxpy as cp
import numpy as np
# (generated code follows)
```
Model Reference (JSON): The following JSON content describes the modeling variables, constraints, and objective function for reference:
{model}

Human

Please generate a Python function for the energy management system using the information below:
- **Available Equipment:** (Description of available devices) {User_information}
- **Equipment Parameters:** (JSON format containing devices specifications) {User_data}
- **State Parameters:** (Dictionary containing the current system state) {User_state}
Use the guidelines from the system prompt to construct the function.

Output

```python
import cvxpy as cp
import numpy as np

def Prosumer(state):
    # Extract state parameters
    T = state['T']
    lambda_B = state['lambda_B']
    lambda_S = state['lambda_S']
    P_Load = state['P_Load']
    Q_Load = state['Q_Load']
    P_RDG1_max = state['P_RDG1_max']
    P_RDG2_max = state['P_RDG2_max']
    SOC_initial_BESS = state['SOC_initial^BESS']
    P_initial_CDG = state['P_initial^CDG']
    parameters
    ...
    # Variables
    P_CDG = cp.Variable(T)
    R_CDG = cp.Variable(T)
    P_Charge_BESS = cp.Variable(T)
    P_Discharge_BESS = cp.Variable(T)
    P_BESS = cp.Variable(T)
    SOC_BESS = cp.Variable(T)
    P_RDG1 = cp.Variable(T)
    Q_RDG1 = cp.Variable(T)
    P_RDG2 = cp.Variable(T)
    Q_RDG2 = cp.Variable(T)
    P_CL = cp.Variable(T)
    P_B = cp.Variable(T)
    P_S = cp.Variable(T)
    Q_B = cp.Variable(T)
    P_EX = cp.Variable(T)
    Q_EX = cp.Variable(T)

    # Constraints
    constraints = []

    # CDG constraints
    constraints.append(P_CDG >= P_min_CDG)
    constraints.append(P_CDG <= P_max_CDG)

    constraints.append(P_CDG[1:] == P_CDG[:-1] + R_CDG[1:])
    constraints.append(P_CDG[0] == P_initial_CDG + R_CDG[0])

    constraints.append(R_CDG >= R_min_CDG)
    constraints.append(R_CDG <= R_max_CDG)

    # BESS constraints
    constraints.append(P_Charge_BESS >= 0)
    constraints.append(P_Charge_BESS <= P_max_BESS)

    constraints.append(P_Discharge_BESS >= 0)
    constraints.append(P_Discharge_BESS <= P_max_BESS)

    constraints.append(SOC_BESS >= SOC_min_BESS)
    constraints.append(SOC_BESS <= SOC_max_BESS)

    constraints.append(P_BESS == eta * P_Charge_BESS - P_Discharge_BESS / eta)

    constraints.append(SOC_BESS[1:] == SOC_BESS[:-1] + P_BESS[1:])
    constraints.append(SOC_BESS[0] == SOC_initial_BESS + eta * P_Charge_BESS[0] - P_Discharge_BESS[0] / eta)

    # CL constraints
    constraints.append(P_CL >= 0)
    constraints.append(P_CL <= alpha * P_Load)

    # RDG1 constraints
    constraints.append(P_RDG1 >= 0)
    constraints.append(P_RDG1 <= P_RDG1_max)
    constraints.append(cp.square(P_RDG1) + cp.square(Q_RDG1) <= S_max_RDG1**2)

    # RDG2 constraints
    constraints.append(P_RDG2 >= 0)
    constraints.append(P_RDG2 <= P_RDG2_max)
    constraints.append(cp.square(P_RDG2) + cp.square(Q_RDG2) <= S_max_RDG2**2)

    # Power purchase/selling constraints
    constraints.append(P_B >= 0)
    constraints.append(P_S >= 0)

    # Power balance constraints (without P2P)
    constraints.append(P_Load + P_BESS - P_CDG - P_RDG1 - P_RDG2 - P_CL - P_B + P_S == 0)
    constraints.append(Q_Load - Q_RDG1 - Q_RDG2 - Q_B == 0)

    # Grid exchange constraints (without P2P)
    constraints.append(P_EX == P_S - P_B)
    constraints.append(Q_EX == -Q_B)

    # Objective function
    C_EX = cp.sum(cp.multiply(lambda_B, P_B) - cp.multiply(lambda_S, P_S))
    C_CDG = cp.sum(a * cp.square(P_CDG) + b * P_CDG + c)
    C_BESS = cp.sum(gamma * (P_Charge_BESS + P_Discharge_BESS))
    C_CL = cp.sum(rho * P_CL)

    objective = cp.Minimize(C_EX + C_CDG + C_BESS + C_CL)

    # Problem formulation and solving
    problem = cp.Problem(objective, constraints)
    problem.solve(solver=cp.GUROBI)
```

(Iterative Correction Expert)

System

You are an expert code corrector specializing in multi-agent microgrid energy management. Your task is to **fix the provided code so that it runs successfully**, please list the things you need to pay attention to before writing the code:
- **Libraries:** Ensure the code uses only `cvxpy` and `numpy`; no other libraries should be used.
- **Units & Values:** All units and values are correct, do not perform any calculations
- **Solver:** Use the Gurobi solver within `cvxpy` for optimization.
- **Excluded Variables:** Confirm that there are no peer-to-peer (P2P) active power variables or constraints.
- **Objective:** The objective should use `cp.Minimize(...)` to minimize total cost.
- **State Dictionary:** Check that the function processes the `state` dictionary correctly and preserves its structure.
- **Function Structure:** Ensure all code is inside `def Prosumer(state):` and the function does not return or print.
- **Encapsulation:** All parameters from the input should be embedded within the function.
- **Strict Compliance:** Do not simplify, assume, or add any formulas or variables that are not in the provided documentation.
- **Complete Code:** Please provide the complete code directly. Refrain from asking for confirmation, generating partial code, or offering any usage examples.

Ensure the corrected code includes:
- **Correct Devices:** Write variables, constraints, and objective functions based on the available devices provided by the user. For devices not provided, please remove the corresponding content from the variables, constraints, and objective functions
- **Correct Parameter Values:** Extract values accurately from the provided documentation and state.
- **Appropriate Constraints:** Including constraints on user available devices in the JSON document.
- **Proper Objective Function:** Formulate the optimization objective correctly to minimize costs.

Modeling requirements for CVX toolbox:
(1).Use ``*`` for matrix-scalar and vector-scalar multiplication.
(2).Use ``@`` for matrix-matrix and matrix-vector multiplication.
(3).Use ``cvxpy.multiply`` for elementwise multiplication.
(4).Please find the appropriate atomic function according to your needs and use ``cvxpy.name`` for atomic function
atomic function:
{tools}
Output Format: The output should be Python code starting with:
```python
import cvxpy as cp
import numpy as np
# (corrected code follows)
```
- **Model Reference (JSON):** Use the following JSON content describing variables, constraints, and objective as guidance:
{model}
- **Structure:** Present the error message and code clearly with labels.
- **Bullet Points:** Use bullet points and bold labels for easy parsing.
- **Code Block:** Include the current code in a code block for clarity.

Human

Please correct the code below based on the information provided:
- **Error message:**
Solution may be inaccurate. Try another solver, adjusting the solver settings, or solve with verbose=True for more information.
- **Current code:**
{code}
- **Available Equipment:**(Description of available devices)
{User_information}
- **Equipment Parameters: **(JSON format containing devices specifications)
{User_data}
- **State Parameters: **(Dictionary containing the current system state)
{User_state}
- **Use the system prompt guidelines to correct the code.

Output

Looking at the error message, the solver is having trouble finding an accurate solution. I'll revise the code to improve numerical stability and solver performance.

Key issues to address:
- **Solver settings:** Need to add proper solver options for better accuracy
- **Numerical stability:** Some constraints might need reformulation for better numerical properties
- **Constraint formulation:** Ensure all constraints are correctly implemented according to the JSON model

Here's the corrected code:

```python
import cvxpy as cp
import numpy as np

def Prosumer(state):
    # Extract state parameters
    T = state['T']
    lambda_B = state['lambda_B']
    lambda_S = state['lambda_S']
    P_Load = state['P_Load']
    Q_Load = state['Q_Load']
    P_RDG1_max = state['P_RDG1_max']
    P_RDG2_max = state['P_RDG2_max']
    SOC_initial_BESS = state['SOC_initial^BESS']
    P_initial_CDG = state['P_initial^CDG']

    # parameters
    ...

    # Variables
    P_CDG = cp.Variable(T, name='P_CDG')
    R_CDG = cp.Variable(T, name='R_CDG')
    P_Charge_BESS = cp.Variable(T, name='P_Charge_BESS')
    P_Discharge_BESS = cp.Variable(T, name='P_Discharge_BESS')
    P_BESS = cp.Variable(T, name='P_BESS')
    SOC_BESS = cp.Variable(T, name='SOC_BESS')
    P_RDG1 = cp.Variable(T, name='P_RDG1')
    Q_RDG1 = cp.Variable(T, name='Q_RDG1')
    P_RDG2 = cp.Variable(T, name='P_RDG2')
    Q_RDG2 = cp.Variable(T, name='Q_RDG2')
    P_CL = cp.Variable(T, name='P_CL')
    P_B = cp.Variable(T, name='P_B')
    P_S = cp.Variable(T, name='P_S')
    Q_B = cp.Variable(T, name='Q_B')
    P_EX = cp.Variable(T, name='P_EX')
    Q_EX = cp.Variable(T, name='Q_EX')

    # Constraints
    constraints = []

    # CDG constraints
    constraints.append(P_CDG >= P_min_CDG)
    constraints.append(P_CDG <= P_max_CDG)

    for t in range(1, T):
        constraints.append(P_CDG[t] == P_CDG[t-1] + R_CDG[t])
    constraints.append(P_CDG[0] == P_initial_CDG + R_CDG[0])

    constraints.append(R_CDG >= R_min_CDG)
    constraints.append(R_CDG <= R_max_CDG)

    # BESS constraints
    constraints.append(P_Charge_BESS >= 0)
    constraints.append(P_Charge_BESS <= P_max_BESS)

    constraints.append(P_Discharge_BESS >= 0)
    constraints.append(P_Discharge_BESS <= P_max_BESS)

    constraints.append(SOC_BESS >= SOC_min_BESS)
    constraints.append(SOC_BESS <= SOC_max_BESS)

    for t in range(T):
        constraints.append(P_BESS[t] == eta * P_Charge_BESS[t] - P_Discharge_BESS[t] / eta)

    for t in range(1, T):
        constraints.append(SOC_BESS[t] == SOC_BESS[t-1] + P_BESS[t])
    constraints.append(SOC_BESS[0] == SOC_initial_BESS - P_Discharge_BESS[0] / eta + eta * P_Charge_BESS[0])

    # CL constraints
    constraints.append(P_CL >= 0)
    for t in range(T):
        constraints.append(P_CL[t] <= alpha * P_Load[t])

    # RDG1 constraints
    constraints.append(P_RDG1 >= 0)
    for t in range(T):
        constraints.append(P_RDG1[t] <= P_RDG1_max[t])
    for t in range(T):
        constraints.append(cp.square(P_RDG1[t]) + cp.square(Q_RDG1[t]) <= S_max_RDG1**2)

    # RDG2 constraints
    constraints.append(P_RDG2 >= 0)
    for t in range(T):
        constraints.append(P_RDG2[t] <= P_RDG2_max[t])
    for t in range(T):
        constraints.append(cp.square(P_RDG2[t]) + cp.square(Q_RDG2[t]) <= S_max_RDG2**2)

    # Power purchase/selling constraints
    constraints.append(P_B >= 0)
    constraints.append(P_S >= 0)

    # Power balance constraints
    for t in range(T):
        constraints.append(P_Load[t] + P_BESS[t] - P_CDG[t] - P_RDG1[t] - P_RDG2[t] - P_CL[t] - P_B[t] + P_S[t] == 0)

    for t in range(T):
        constraints.append(Q_Load[t] - Q_RDG1[t] - Q_RDG2[t] - Q_B[t] == 0)

    # Grid exchange constraints
    for t in range(T):
        constraints.append(P_EX[t] == P_S[t] - P_B[t])

    for t in range(T):
        constraints.append(Q_EX[t] == -Q_B[t])

    # Objective function
    C_EX = cp.sum(cp.multiply(lambda_B, P_B) - cp.multiply(lambda_S, P_S))
    C_CDG = cp.sum(a * cp.square(P_CDG) + b * P_CDG + c)
    C_BESS = cp.sum(gamma * (P_Charge_BESS + P_Discharge_BESS))
    C_CL = cp.sum(rho * P_CL)

    objective = cp.Minimize(C_EX + C_CDG + C_BESS + C_CL)

    # Problem formulation and solving with improved solver settings
    problem = cp.Problem(objective, constraints)
    problem.solve(solver=cp.GUROBI, verbose=False,
                  BarConvTol=1e-9,
                  FeasibilityTol=1e-9,
                  OptimalityTol=1e-9,
                  NumericFocus=3)
```

(Energy Trading Integration Expert)

System

You are an expert code reviewer for multi-agent microgrid energy management systems. Your task is to **modify the provided code by adding or adjusting specified elements** to ensure it executes correctly. Proceed step by step, addressing each of the following points:
- **Add P2P Variables:** Add peer-to-peer (P2P) active power variables as specified in the JSON model if they are not already present.
- **Balance Constraints:** Include the new P2P active power variables in the active power balance constraint (as per the JSON model) and in the grid-exchange power constraint, ensuring total balance.
- **Action Dictionary:** Create a dictionary named `action` containing references to relevant variables. For example:
  action = [
    "P_BESS": P_BESS if 'P_BESS' in variables else 0,
    "R_CDG": R_CDG if 'R_CDG' in variables else 0,
    "P_CL": P_CL if 'P_CL' in variables else 0,
    "P_RDG1": P_RDG1 if 'P_RDG1' in variables else 0,
    "Q_RDG1": Q_RDG1 if 'Q_RDG1' in variables else 0
  ](Adjust variable names as needed and exclude any non-existent entries.)
- **Return Statement:** Add a return statement that returns exactly 6 elements in this order(do not lack any return content below):
  1.The action dictionary.
  2.The objective function expression without cp.Minimize (just the expression).
  3.A list of constraints.
  4.The grid-exchange active power variable P_EX_t (cvxpy variable of dimension T).
  5.The grid-exchange reactive power variable Q_EX_t (cvxpy variable of dimension T).
  6.The P2P transaction active power variable P_P2P_t (cvxpy variable of dimension T).
- **Remove Solver Code:** Remove any solver configuration lines (e.g. Problem.set_solver(GUROBI)) and any calls to problem.solve().
- **Remove cp.Minimize/Maximize:** Ensure the objective is defined without wrapping it in cp.Minimize or cp.Maximize.
- **Do Not Alter Other Parts:** Do not change any input variables or other existing code except as needed for these modifications.
- **Apply Changes Directly:** Please provide the update code directly. Refrain from asking for confirmation, generating partial code, or offering any usage examples.
- **Output Format:** The output should be Python code with the following imports at the top:
  ```python
  import cvxpy as cp
  import numpy as np
  #The code you generated
  ```
- **Model Reference (JSON):** The following JSON content shows the modeling variables, constraints, and objective for reference:
{context_docs}

Human

Below is the current code that needs revision:
{code}
Apply the revision instructions from the system prompt to this code.

Output

```python
import cvxpy as cp
import numpy as np

def Prosumer(state):
    # Extract state parameters
    T = state['T']
    lambda_B = state['lambda_B']
    lambda_S = state['lambda_S']
    P_Load = state['P_Load']
    Q_Load = state['Q_Load']
    P_RDG1_max = state['P_RDG1_max']
    P_RDG2_max = state['P_RDG2_max']
    SOC_initial_BESS = state['SOC_initial^BESS']
    P_initial_CDG = state['P_initial^CDG']

    # CDG parameters
    ...

    # Variables
    P_CDG = cp.Variable(T, name='P_CDG')
    R_CDG = cp.Variable(T, name='R_CDG')
    P_Charge_BESS = cp.Variable(T, name='P_Charge_BESS')
    P_Discharge_BESS = cp.Variable(T, name='P_Discharge_BESS')
    P_BESS = cp.Variable(T, name='P_BESS')
    SOC_BESS = cp.Variable(T, name='SOC_BESS')
    P_RDG1 = cp.Variable(T, name='P_RDG1')
    Q_RDG1 = cp.Variable(T, name='Q_RDG1')
    P_RDG2 = cp.Variable(T, name='P_RDG2')
    Q_RDG2 = cp.Variable(T, name='Q_RDG2')
    P_CL = cp.Variable(T, name='P_CL')
    P_B = cp.Variable(T, name='P_B')
    P_S = cp.Variable(T, name='P_S')
    Q_B = cp.Variable(T, name='Q_B')
    P_P2P = cp.Variable(T, name='P_P2P')
    P_EX = cp.Variable(T, name='P_EX')
    Q_EX = cp.Variable(T, name='Q_EX')

    # Constraints
    constraints = []

    # CDG constraints
    constraints.append(P_CDG >= P_min_CDG)
    constraints.append(P_CDG <= P_max_CDG)

    for t in range(1, T):
        constraints.append(P_CDG[t] == P_CDG[t-1] + R_CDG[t])
    constraints.append(P_CDG[0] == P_initial_CDG + R_CDG[0])

    constraints.append(R_CDG >= R_min_CDG)
    constraints.append(R_CDG <= R_max_CDG)

    # BESS constraints
    constraints.append(P_Charge_BESS >= 0)
    constraints.append(P_Charge_BESS <= P_max_BESS)

    constraints.append(P_Discharge_BESS >= 0)
    constraints.append(P_Discharge_BESS <= P_max_BESS)

    constraints.append(SOC_BESS >= SOC_min_BESS)
    constraints.append(SOC_BESS <= SOC_max_BESS)

    for t in range(T):
        constraints.append(P_BESS[t] == eta * P_Charge_BESS[t] - P_Discharge_BESS[t] / eta)

    for t in range(1, T):
        constraints.append(SOC_BESS[t] == SOC_BESS[t-1] + P_BESS[t])
    constraints.append(SOC_BESS[0] == SOC_initial_BESS - P_Discharge_BESS[0] / eta + eta * P_Charge_BESS[0])

    # CL constraints
    constraints.append(P_CL >= 0)
    for t in range(T):
        constraints.append(P_CL[t] <= alpha * P_Load[t])

    # RDG1 constraints
    constraints.append(P_RDG1 >= 0)
    for t in range(T):
        constraints.append(P_RDG1[t] <= P_RDG1_max[t])
    for t in range(T):
        constraints.append(cp.square(P_RDG1[t]) + cp.square(Q_RDG1[t]) <= S_max_RDG1**2)

    # RDG2 constraints
    constraints.append(P_RDG2 >= 0)
    for t in range(T):
        constraints.append(P_RDG2[t] <= P_RDG2_max[t])
    for t in range(T):
        constraints.append(cp.square(P_RDG2[t]) + cp.square(Q_RDG2[t]) <= S_max_RDG2**2)

    # Power purchase/selling constraints
    constraints.append(P_B >= 0)
    constraints.append(P_S >= 0)

    # Power balance constraints (including P2P)
    for t in range(T):
        constraints.append(P_Load[t] + P_BESS[t] - P_CDG[t] - P_RDG1[t] - P_RDG2[t] - P_CL[t] - P_B[t] - P_P2P[t] + P_S[t] == 0)

    for t in range(T):
        constraints.append(Q_Load[t] - Q_RDG1[t] - Q_RDG2[t] - Q_B[t] == 0)

    # Grid exchange constraints (including P2P)
    for t in range(T):
        constraints.append(P_EX[t] == P_S[t] - P_B[t] - P_P2P[t])

    for t in range(T):
        constraints.append(Q_EX[t] == -Q_B[t])

    # Objective function
    C_EX = cp.sum(cp.multiply(lambda_B, P_B) - cp.multiply(lambda_S, P_S))
    C_CDG = cp.sum(a * cp.square(P_CDG) + b * P_CDG + c)
    C_BESS = cp.sum(gamma * (P_Charge_BESS + P_Discharge_BESS))
    C_CL = cp.sum(rho * P_CL)

    objective = C_EX + C_CDG + C_BESS + C_CL

    # Create action dictionary
    action = {
        "P_BESS": P_BESS,
        "R_CDG": R_CDG,
        "P_CL": P_CL,
        "P_RDG1": P_RDG1,
        "Q_RDG1": Q_RDG1,
        "P_RDG2": P_RDG2,
        "Q_RDG2": Q_RDG2
    }

    return action, objective, constraints, P_EX, Q_EX, P_P2P
```